



Estudo de **Design Patterns** aplicados ao ecossistema

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 02 · Práticas de Engenharia e Clean Code

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

Adapter / Strategy / Observer

1. Contexto

A FV precisa plugar fornecedores diferentes (meios de pagamento, transportadoras, gateways de SMS) sem reescrever o core a cada contratação. O código legado tinha um `if fornecedor == 'X'` para cada integração, e o time temia qualquer onboarding de parceiro novo. A **flexibilidade** era paga em complexidade.

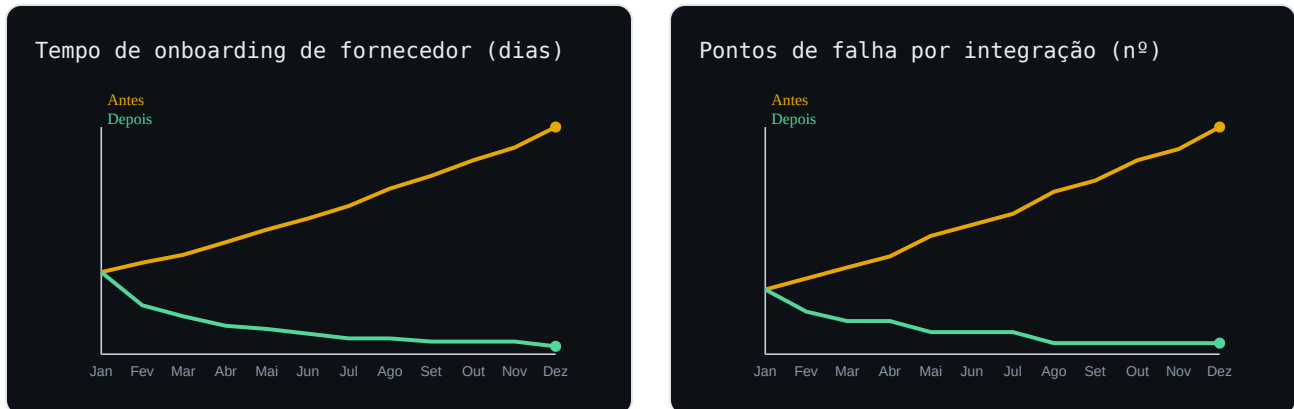
ANALOGIA DOS ADAPTADORES DE TOMADA

Viajar pelo mundo é fácil porque existem **adaptadores de tomada**: o seu carregador (core) não muda, só o pedacinho que encaixa na parede (fornecedor) muda.

Strategy é escolher o melhor caminho sem mudar o destino; *Observer* é o painel que avisa todo mundo quando algo acontece. Padrões são esses 'encaixes' reutilizáveis.

2. Diagnóstico

Com lógica de integração espalhada em condicionais, o tempo de onboarding de fornecedor cresce e cada novo parceiro duplica pontos de falha. O core se polui com detalhes de terceiros e fica mais difícil de testar.



Raiz do problema: o core conhece os detalhes de cada fornecedor. Falta uma *camada de indireção* que isole variação atrás de uma interface estável.

3. Solução

Aplicar três **Design Patterns** na FV: **Adapter** (traduz a API de terceiro para a interface do core: ports/adapters), **Strategy** (seleciona em runtime o algoritmo/fornecedor ideal) e **Observer** (notifica partes interessadas quando um evento de domínio ocorre, sem acoplar).

Adapter: o core fala uma língua só:

```
# port do core
class GatewayPagamento:
    def cobrar(self, valor: float) -> str: ...

# adapter traduz o PayPal para o port
class PayPalAdapter(GatewayPagamento):
    def cobrar(self, valor):
        return api_paypal.charge(amount=valor) # detalhe isolado
```

Strategy: escolhe o melhor frete em runtime:

```
class FreteStrategy:
    def calcular(self, pedido): raise NotImplementedError
```

```
class FreteExpresso(FreteStrategy):
    def calcular(self, p): return p.peso * 2.0
class FreteEconomico(FreteStrategy):
    def calcular(self, p): return p.peso * 0.8

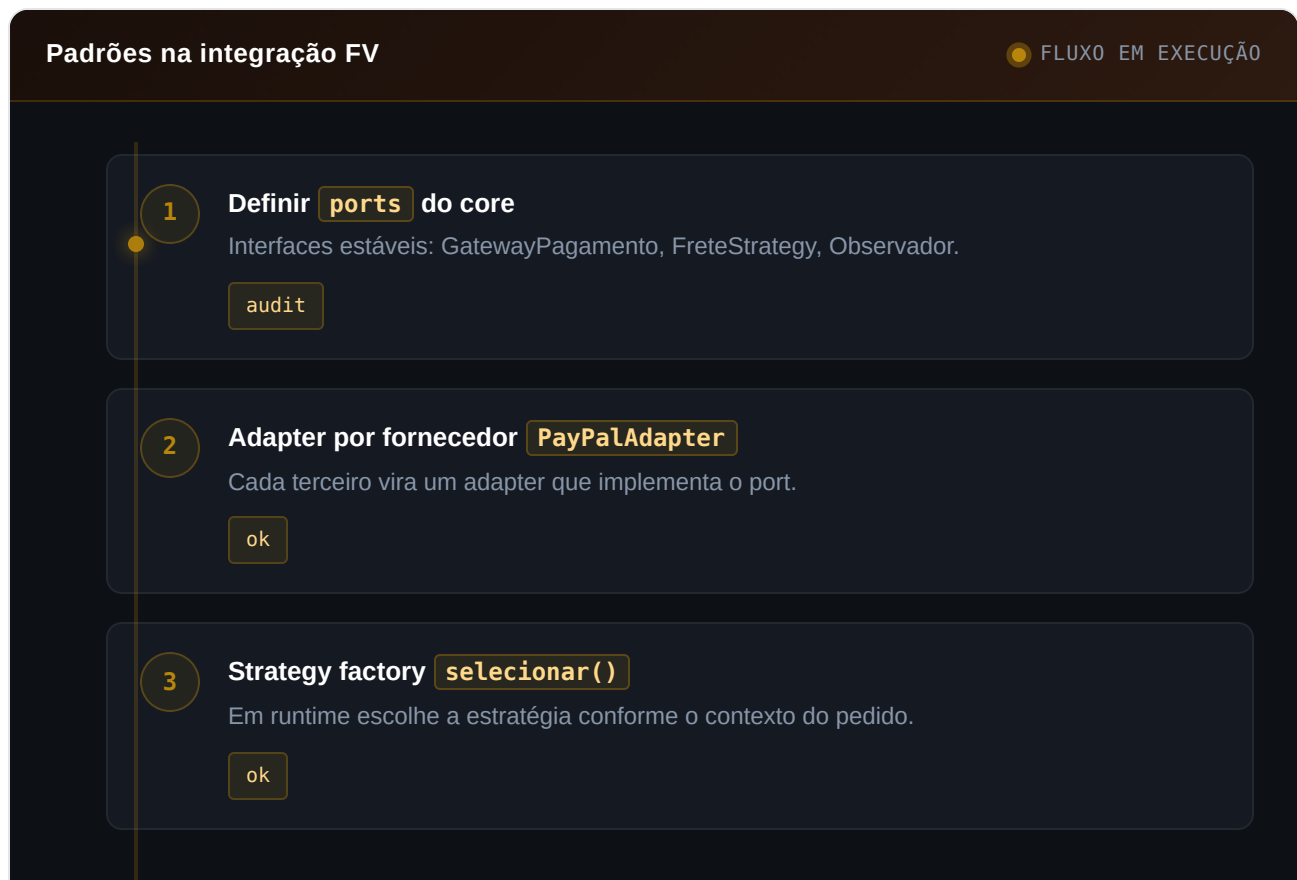
def selecionar(pedido) -> FreteStrategy: # Strategy factory
    return FreteExpresso() if pedido.urgente else FreteEconomico()
```

Observer: o core avisa, os interessados reagem:

```
class Observador: # Observer
    def ao_pedido_pago(self, ev): ...
class NotificaCliente(Observador):
    def ao_pedido_pago(self, ev): email.enviar(ev.pedido_id)
class AtualizaEstoque(Observador):
    def ao_pedido_pago(self, ev): estoque.debitar(ev.itens)

bus.assinar(PedidoPago, NotificaCliente())
bus.assinar(PedidoPago, AtualizaEstoque())
```

4. Como funciona (pipeline)



4

Observer `bus.assinar()`
Reações (e-mail, estoque) assinam eventos sem acoplar ao core.
`warn`

5

Testes com fakes
Port fake valida o core sem chamar fornecedor de verdade.
`ok`

Novo fornecedor = novo adapter; o core sequer recompila.

5. Antes vs Depois

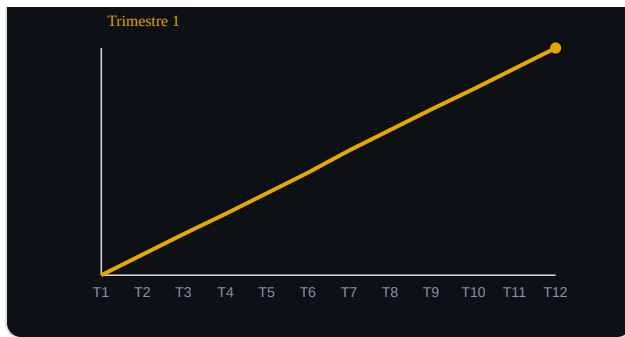
CENÁRIO	ANTES (IF/ELSE)	DEPOIS (PADRÕES)
Novo fornecedor	Mexe no core todo	Só adapter novo
Trocar frete	Reescreve fluxo	Strategy em runtime
Avisar parceiro	Acopla no meio	Observer assina
Testar core	Precisa terceiro	Fake do port

6. Entregas

- **Pacote** adapters/: PayPalAdapter, StripeAdapter, CorreiosAdapter.
- **Pacote** strategies/: FreteExpresso, FreteEconomico + factory.
- **Pacote** observers/: NotificaCliente, AtualizaEstoque, EmiteNota.
- **Exemplo** onboarding_fornecedor.md: como plugar um novo adapter.

7. Métricas

Velocidade de plugar fornecedor
(fornecedores/mês)



Meta: onboarding de novo fornecedor em ≤ 2 dias, atrás de adapter testado, sem tocar no core.

9. Decisões e tradeoffs

- Adapter para CRM e LLM externos: isola o acoplamento a APIs de terceiro em tradutores atrás de ports, então novo fornecedor vira um adapter novo sem tocar o core.
- Strategy para roteamento de modelo de LLM e cálculo variável: troca if e else espalhados por estratégias selecionadas em runtime via factory selecionar.
- Observer para eventos de domínio com bus.assinar: reações como e-mail e estoque assinam eventos sem acoplar ao core, aceitando ordem não garantida entre observadores.
- Singleton para clients rejeitado em favor de DI: evita estado global oculto e mantém o core testável com fakes dos ports.
- Padrão aplicado só onde há variação real, com code review focado em valor: evita over-engineering e impede que padrão vire fim em si mesmo.

10. Impacto no negócio

Handlers de webhook e workers repetitivos com seleção de modelo por if e else fazem cada onboarding de fornecedor duplicar pontos de falha e travar o time. Com Adapter, Strategy e Observer mais piloto em ao menos 1 CRM, o onboarding cai para até 2 dias atrás de adapter testado, o teste do core usa fake sem chamar terceiro, e o custo de plugar parceiro novo deixa de ser reescrita do fluxo.

11. Referências de estudo

- Curso: Design Patterns com Python, na Alura.
- Vídeo: Strategy na prática para trocar if else, no YouTube.
- Doc oficial: Catálogo de padrões com exemplos em Python, em refactoring.guru.

- Doc oficial: Documentação do Python sobre abc e protocolos para ports e adapters, em docs.python.org.

Nota de autoria: seções de retrofit derivadas do conteúdo já existente na atividade, sem métricas novas e sem links não verificados.

8. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.